

An Explainable Multi-Cloud FinOps Decision Engine: Canonical Cost Normalization and Lifecycle-Aware Recommendation Support

Keerthi Rapolu

Rishika Naha
Independent Research

Abstract—This paper presents Multi-Cloud FinOps Decision Engine, a reproducible post-billing FinOps framework that converts heterogeneous AWS, Azure, and GCP billing exports into canonical cost, allocation, and decision artifacts. The system combines a Unified Cost Allocation Schema, cloud-specific Net Effective Cost (NEC) modeling, shared-cost distribution, deterministic waste detection, structured reasoning over billing signals, auditable recommendation scoring, lightweight month-end forecasting, and lifecycle-aware validation outputs. Evaluated on a synthetic benchmark with 18,992 normalized billing rows, the implementation materializes 95 persisted recommendations and achieves forecast backtest error of \$7.86 mean absolute error and 2.98% mean absolute percentage error. The results show that cross-cloud billing semantics can be normalized into an explainable decision pipeline without relying on proprietary vendor tooling, while also making the current validation boundary explicit: the artifact strongly supports reproducibility, traceability, and internal consistency, but lifecycle realization evidence remains pre-production.

Index Terms—FinOps, multi-cloud, cost allocation, net effective cost, decision intelligence, cloud cost optimization

I. INTRODUCTION

FinOps practice becomes materially harder once cost governance spans multiple cloud providers. AWS, Azure, and GCP each expose different billing exports, discount semantics, commitment accounting behaviors, and ownership signals [1], [2], [3]. A team that looks inexpensive in one raw export can appear expensive in another simply because reservation amortization, savings-plan effective cost, or credit handling is encoded differently. The first systems challenge in multi-cloud optimization is therefore not recommendation ranking or dashboard design; it is establishing a cost model that makes cross-cloud comparison economically and operationally defensible.

This problem is especially acute for post-billing decision support. Raw billing data mixes consumed workload cost with idle commitment waste, hides responsibility behind incomplete tags or shared subscriptions, and often collapses distinct savings notions into one unsafe number. In the current benchmark state of this repository, the unified billing mart contains 18,992 normalized rows across AWS, Azure, and GCP, with \$52,021.31 in net effective cost (NEC) and \$29,874.07 of unattributed NEC. These values are useful precisely because

they are derived from an explicit canonical model rather than from provider-local billing conventions.

Most industry tooling provides descriptive visibility first: spend breakdowns, trend charts, and vendor-specific optimization hints. Those views are useful, but they are not enough for auditable multi-cloud decision making. A systems paper must instead answer four harder questions: how heterogeneous billing exports are normalized into one data contract, how commitment-aware effective cost is computed consistently, how shared and unattributed spend become accountable optimization scope, and how recommendations, forecasts, and validation outputs are generated transparently from that canonical substrate.

This paper presents a reproducible post-billing *multi-cloud FinOps decision engine* rather than a dashboard-only cost viewer. The system ingests provider billing exports, computes cloud-specific NEC semantics, materializes a Unified Cost Allocation Schema (CAS), allocates shared spend into accountable scope, detects optimization and governance issues, ranks recommendations with auditable scoring, forecasts month-end NEC, and persists lifecycle-aware validation outputs. All stages are implemented as reproducible repository artifacts over a synthetic multi-cloud benchmark, allowing the paper to make concrete design and evaluation claims without depending on proprietary enterprise billing data.

Contributions

The paper makes four primary contributions:

- It defines and implements a canonical Unified Cost Allocation Schema (CAS) that normalizes heterogeneous AWS, Azure, and GCP billing semantics into one downstream data contract for attribution, NEC analysis, and decision generation.
- It introduces a commitment-aware NEC modeling layer that separates consumed workload cost from commitment waste and avoids the failure modes of naive billing fields.
- It presents an auditable decision pipeline that connects waste detection, structured reasoning over billing facts, explicit signal-to-action mapping, deterministic recommendation scoring, and lightweight month-end forecasting.

- It demonstrates the system on a reproducible synthetic benchmark, yielding 95 materialized recommendations and forecast backtests with \$7.86 mean absolute error, 2.98% mean absolute percentage error, and a 95.59% within-10-percent rate.

Two motivating examples clarify why this framing matters. First, RI-covered or Savings Plan-covered AWS usage can appear misleadingly cheap or free in raw fields, even though the enterprise has already incurred commitment cost. Second, tag gaps and shared-cost rows can block accountable optimization even when total spend is visible; the current benchmark’s \$29,874.07 in unattributed NEC shows that visibility without attribution is not sufficient for action.

The remainder of the paper is organized as follows. Section III defines the Unified Cost Allocation Schema and its cross-cloud mapping decisions. Section IV explains shared-cost allocation and accountable cost propagation. Section V formalizes NEC modeling and validation. Section VI presents the decision-intelligence pipeline, including waste detection, recommendation scoring, forecasting, and lifecycle validation. Section VII summarizes implementation details, benchmark construction, and evaluation evidence. Section VIII concludes with limitations and future work.

II. BACKGROUND AND RELATED WORK

FinOps has matured into an operational framework for technology value management that emphasizes collaboration between engineering, finance, and business stakeholders, with iterative phases commonly described as Inform, Optimize, and Operate [4], [5]. That framing is useful for this paper because cost management is not merely retrospective reporting. It requires timely data, attribution structures, optimization logic, and a feedback path from proposed action to realized outcome.

A. Multi-Cloud Cost and Attribution Context

Prior work on chargeback for cloud services argues that accountability, transparency, and measurability are central to successful cloud cost allocation [6]. Those concerns become sharper in multi-cloud settings, where billing and SLA semantics differ across providers and make manual comparison difficult [7], [8]. In practice, multi-cloud consumers must reconcile provider-specific exports, normalize ownership and pricing fields, and decide how to treat shared or unattributed spend before any optimization recommendation can be trusted.

B. Provider-Native Billing Semantics

The major providers already expose rich billing exports, but they do so with different semantics. AWS Cost and Usage Reports distinguish line-item types such as `Usage`, `DiscountedUsage`, `SavingsPlanCoveredUsage`, `RIFee`, and `SavingsPlanRecurringFee`, and also provide reservation-effective-cost columns that are populated only for certain row types [1], [9]. Azure Cost Management distinguishes actual and amortized cost views and surfaces explicit unused commitment charge types such as `UnusedReservation` and `UnusedSavingsPlan`

only in amortized analysis [2], [10]. Google Cloud Billing exports attach discounts and credits directly to usage records and document how committed-use discounts appear through commitment fees, effective prices, and credit offsets [11], [3]. These native exports are necessary inputs, but they are not themselves a cross-cloud analytical contract.

C. Standards and Differentiation

The FinOps Open Cost and Usage Specification (FOCUS) is an important standardization effort for cross-provider billing data [12]. It reduces naming friction and encourages a common schema vocabulary, but it does not by itself define cloud-specific NEC formulas, shared-cost allocation behavior, recommendation logic, or lifecycle validation outputs. Existing literature also contains broad surveys of cloud cost optimization strategies [13], yet much of that work focuses on provider-local optimization rather than on an implemented end-to-end decision pipeline for post-billing FinOps.

This paper therefore positions its contribution between provider-native documentation and abstract optimization literature. Relative to native tooling, it contributes an open and reproducible cross-cloud decision substrate. Relative to billing standards, it contributes explicit NEC, allocation, and validation semantics. Relative to prior research on chargeback and multi-cloud management, it contributes an implementation-backed pipeline that links normalization, attribution, signals, recommendations, forecasting, and lifecycle-aware evaluation in one auditable system.

III. UNIFIED COST ALLOCATION SCHEMA DESIGN

The Unified Cost Allocation Schema (CAS) is the paper’s primary normalization layer. Its role is to decouple downstream analysis from provider-specific billing exports by defining one canonical contract for cost identity, pricing, ownership, allocation state, and decision features. Without such a contract, any cross-cloud recommendation layer is forced either to duplicate provider-specific logic in every downstream component or to operate on economically inconsistent fields.

The implemented CAS groups fields into a small number of stable families: identity, pricing, resource description, ownership, and allocation. Identity fields capture cloud, billing month, account or subscription scope, and service category. Pricing fields carry list cost, effective cost, and commitment-related waste terms. Resource fields preserve service, SKU, and resource labels needed for drill-down and recommendation explanation. Ownership and allocation fields record direct tags, inferred team scope, and shared-cost allocation outcomes.

CAS design is shaped by two cross-cloud mismatches. First, the three providers encode effective cost differently: AWS distributes commitment economics through line-item types and effective-cost columns, Azure distinguishes actual versus amortized analysis, and GCP blends usage, fees, credits, and discount semantics across export fields. Second, ownership signals vary in completeness and naming quality, so the schema must preserve both direct attribution and unresolved accountability gaps rather than forcing false precision.

TABLE I
CAS FIELD FAMILIES.

Family	Representative fields	Purpose
Identity	cloud, billing month, billing scope	Stable grouping keys across providers
Pricing	list cost, NEC, commitment waste	Economically comparable cost analysis
Resource	service, SKU, resource label	Explainable drill-down and recommendation context
Ownership	tag owner, team, environment	Accountability and governance analysis
Allocation	shared flag, allocated team, basis	Explicit post-allocation reporting state

The mapping decisions are intentionally conservative. CAS normalizes names and types, but it does not erase provider-specific semantics that matter economically. Instead, provider-local transformation models compute cloud-specific NEC terms first and then surface the results through a common schema. This layered design keeps the canonical contract stable while isolating schema drift and provider-specific billing changes to a narrower transformation boundary.

Schema evolution is therefore a first-class design concern. New provider fields should either map into an existing family, extend a provider-local staging model, or be added as optional canonical attributes only if they materially improve cross-cloud reasoning. This avoids the common failure mode in which a supposedly unified schema becomes an unstable mirror of vendor exports.

IV. COST ALLOCATION STRATEGIES

Cross-cloud optimization is not only a pricing problem; it is also an accountability problem. Many billing rows are directly attributable to a team or workload, but platform subscriptions, shared networking, observability, and other common services often are not. The allocation layer therefore has two goals: preserve the economic total exactly, and project shared or unattributed spend into accountable analytical scope without hiding uncertainty.

The implemented system distinguishes direct allocation from shared-cost allocation. Direct allocation uses explicit ownership signals such as tags, subscription metadata, or team mappings. Shared-cost allocation applies when a cost is intentionally communal and should be fanned out across recipients using a stated basis. Unresolved unattributed spend remains visible rather than being silently forced into a misleading owner bucket.

A. Shared-Cost Distribution

Let C_s denote shared cost and let w_i denote the weight assigned to recipient i such that $\sum_i w_i = 1$. The allocated shared cost for recipient i is

$$A_i = C_s w_i. \quad (1)$$

The core invariant is

$$\sum_i A_i = C_s, \quad (2)$$

TABLE II
ALLOCATION STRATEGIES.

Strategy	Basis	When appropriate
Direct owner	Explicit metadata	Resource already has trustworthy owner tags
Usage share	Team NEC or workload share	Shared platform spend scales with actual consumption
Fixed split	Configured ratios	Organizational or SLA-based chargeback rules
Unresolved gap	No allocation applied	Ownership evidence is too weak for forced assignment

which prevents the allocation layer from creating or destroying cost.

In the current benchmark, Azure shared-cost fan-out preserves totals exactly, with zero delta between pre-allocation shared NEC and allocated shared NEC in every observed month. That result matters because it validates the accounting invariant before recommendation logic consumes the allocation outputs.

B. System Integration

Allocation outputs are treated as first-class decision features rather than as reporting-only artifacts. Team NEC summaries, shared-cost reporting, governance signals, and recommendation ownership all depend on the same allocation state. This design prevents a common operational failure in which dashboards, anomaly detection, and action queues each use a different notion of “owner.” The result is a more defensible bridge from billing visibility to actionability.

V. NET EFFECTIVE COST MODELING

Naive billing fields are not reliable measures of workload economics in a multi-cloud setting. Raw list price overstates committed usage, while certain effective-cost or credit fields can make covered usage appear artificially cheap or free when commitment spend has simply moved elsewhere in the export. NEC modeling addresses this by defining consumed workload cost and commitment waste explicitly before downstream reporting and recommendation logic begin.

We use NEC to mean the economically attributable cost of consumed workload after provider-specific commitment semantics have been normalized. NEC is deliberately separated from explicit waste terms so that actionable workload economics are not conflated with idle commitment loss.

A. Cloud-Specific Definitions

For AWS, NEC must account for on-demand rows, RI-covered rows, Savings Plan-covered rows, and explicit commitment waste rows. In simplified form,

$$NEC_{used}^{AWS} = c_{od} + c_{ri} + c_{sp}, \quad (3)$$

where the three terms denote on-demand, RI-covered, and Savings Plan-covered effective cost components. Explicit AWS commitment waste is

$$NEC_{waste}^{AWS} = w_{ri} + w_{sp}. \quad (4)$$

TABLE III
NEC INTERPRETATION BY CLOUD.

Cloud	Consumed NEC basis	Waste separation
AWS	On-demand plus RI/SP effective cost	Explicit RI and Savings Plan waste rows
Azure	Amortized consumed cost	Unused reservation and savings-plan charges
GCP	Usage cost after discounts and credits	Non-consumed commitment or governance terms handled separately

For Azure, the key distinction is between actual and amortized analysis. The paper uses amortized views when modeling commitment-aware economics, so NEC is derived from amortized consumed cost while unused reservation and unused savings-plan rows are separated as waste terms.

For GCP, exported discounts and credits are attached more directly to usage records. A simplified consumed-cost formulation is

$$NEC_{used}^{GCP} = \max(c_{list} + c_{credit}, 0), \quad (5)$$

with commitment fees and other non-consumed terms handled separately when they represent waste or non-workload charges.

B. Validation and Benchmark Evidence

The current benchmark reports aggregate list cost of \$67,065.52 and aggregate NEC of \$52,021.31, leaving \$15,044.21 of modeled savings relative to list price and \$136.80 of explicit commitment waste. These results illustrate why NEC should not be collapsed into naive list-cost reporting. The system also uses NEC as the economic base for per-team accountability, recommendation ranking, and forecast generation, so validation of NEC consistency is foundational for the rest of the pipeline.

The implementation validates NEC in two ways. First, cloud-level rollups and downstream marts remain internally consistent after normalization. Second, waste is tracked explicitly rather than being mixed into consumed workload cost. This separation is methodologically important because it prevents reviewer-visible claims of optimization savings from being inflated by accounting artifacts.

VI. DECISION INTELLIGENCE

The central contribution of the second project phase is a deterministic decision pipeline that operates on canonical billing and allocation outputs rather than on dashboard-local heuristics. Starting from the unified billing mart, the system materializes signals, recommendations, forecast outputs, and lifecycle summaries as explicit artifacts. This makes the intelligence layer auditable: every recommendation can be traced back to a normalized billing condition, a fixed mapping rule, and a persisted score decomposition.

A. Waste Detection

The waste detector classifies inefficiency into a finite taxonomy rather than relying on a generic overspend flag. The

TABLE IV
IMPLEMENTED WASTE TAXONOMY.

Waste type	Signal	Conf.	Main limitation
Unused commitment	Explicit commitment waste rows	High	Depends on provider waste rows being exposed correctly
Underutilized commitment	Commitment utilization below configured threshold	Medium	Sensitive to threshold choice
Idle compute proxy	NEC-per-vCPU proxy below configured floor	Medium	Billing-derived proxy, not runtime telemetry
Zombie resource	Near-zero NEC over the observation window	Medium	Low-cost resources may still have latent purpose

implemented categories are unused commitment, underutilized commitment, idle compute proxy, and zombie resource. Thresholds are loaded from configuration, allowing sensitivity to be tuned without changing code.

The distinction between deterministic and proxy signals is important. Unused commitment is close to deterministic because it is derived from explicit waste fields. By contrast, idle compute is a billing-side approximation of low utilization; the system intentionally does not claim runtime observability if it does not have.

B. Structured Reasoning and Recommendation Scoring

The next layer performs structured reasoning over billing-derived facts. Methodologically, it is better understood as a ranked explanation layer over cost signals, trends, and anomalies than as intervention-based causal inference. Fact types such as commitment waste, idle resources, unattributed cost gaps, and service-category dominance are combined into persisted explanatory narratives.

Recommendations are generated from canonical signals using an explicit signal-to-action map. Idle compute proxy findings map to `resize_down`, zombie resource findings map to `remove_resource`, governance gaps map to `enforce_tags`, and forecast risk signals map to `review_forecast_risk`. Recommendation ranking is deterministic:

$$p = 0.35s + 0.25c + 0.20g + 0.10u + 0.10l \quad (6)$$

where p is priority score, s is normalized savings, c is confidence, g is governance severity, u is urgency, and l is a low-risk bonus.

Current benchmark outputs show that the recommendation space is dominated by idle compute proxy actions. Query results report 48 medium-risk `resize_down` recommendations accounting for \$1,545.74 in estimated savings, while governance-oriented `enforce_tags` actions carry zero direct savings but remain important because unattributed NEC blocks accountable optimization.

TABLE V
REPRESENTATIVE SIGNAL-TO-ACTION MAPPINGS.

Signal	Action	Recovery	Interpretation
Idle compute proxy	Resize down	60%	Conservative efficiency recovery from a billing-side idle signal
Zombie resource	Remove resource	100%	Remove a resource when spend is effectively dormant
Tagging gap	Enforce tags	0% direct	Governance action that improves optimization readiness
Unattributed cost gap	Enforce tags	0% direct	Accountability repair rather than immediate cost reduction
Forecast risk	Review forecast risk	0% direct	Early warning rather than direct savings action

C. Forecasting and Lifecycle Validation

The forecasting layer is intentionally lightweight. It estimates month-end NEC using a 70/30 blend of current run rate and trailing three-month average whenever sufficient history exists; otherwise it falls back to current run rate alone:

$$f = 0.7r + 0.3a \quad (7)$$

or, when history is insufficient,

$$f = r. \quad (8)$$

This model favors transparency over predictive complexity. The current benchmark backtest reports 68 evaluations with mean absolute error of \$7.86, mean absolute percentage error of 2.98%, and a within-10-percent rate of 95.59%.

The final layer treats recommendations as persisted operational objects rather than disposable dashboard rows. Lifecycle records store states such as `recommended`, `approved`, `implemented`, and `verified`, along with expected savings, realized savings, and derived accuracy. The current repository state should be described carefully: all 95 recommendations are still in the `recommended` state, with \$1,709.83 expected savings and \$0 realized savings. The lifecycle layer is therefore best presented as implemented validation instrumentation rather than mature realized-savings evidence.

VII. IMPLEMENTATION AND EVALUATION

The repository implements the full pipeline as a reproducible local artifact stack. Python ingestors validate and land provider exports, dbt models materialize staging, intermediate, and mart tables in DuckDB, and downstream Python modules consume the unified mart for allocation, intelligence, forecasting, and lifecycle persistence. The implementation is intentionally transparent rather than distributed: it favors inspectable data products and deterministic rule logic over opaque service orchestration.

A. Benchmark and Reproducibility

Evaluation is performed on a synthetic multi-cloud benchmark that exercises the financial and governance cases the paper claims to handle. The benchmark includes commitment-backed rows, shared-cost scenarios, tag gaps, unattributed spend, anomaly-style month-to-month variation, and lifecycle records. Reproducibility is supported directly in the repository through synthetic data generators, a dbt build path, a DuckDB-backed paper query pack, exported CSV and Markdown result sets, and LaTeX figure-generation scripts. Every headline number in the paper can therefore be regenerated from the checked-in pipeline rather than copied from one-off analysis.

B. Current Benchmark Snapshot

The current database state contains 18,992 normalized rows across four billing months. Aggregate list cost is \$67,065.52 and aggregate NEC is \$52,021.31, leaving \$15,044.21 of savings relative to list price and \$136.80 of explicitly modeled commitment waste. The benchmark also materializes \$29,874.07 of unattributed NEC, which serves as a governance stress case because it forces the system to distinguish visibility from accountability. On the decision side, the repository currently materializes 95 recommendations with \$1,709.83 of expected savings represented in lifecycle rows.

C. Evidence Summary

Several implemented invariants are already supported by current outputs. Shared-cost preservation holds exactly in the Azure benchmark path after fan-out, with zero delta between shared NEC and allocated shared NEC in each observed month. NEC summaries are internally consistent across cloud-level rollups and commitment-utilization measures. Recommendation outputs are auditable back to signal types, action mappings, and score components. Forecast quality is summarized by 68 backtest evaluations with \$7.86 mean absolute error, 2.98% mean absolute percentage error, and a 95.59% within-10-percent rate. The lifecycle layer is implemented end to end, but the benchmark should still be described as pre-realization: recommendations are persisted and trackable, yet realized-savings evidence remains sparse because current rows are concentrated in the `recommended` state.

D. Scope and Limitations

The evaluation is intentionally bounded. The benchmark is synthetic rather than enterprise production billing data. Waste detection relies on billing-derived proxies and commitment semantics rather than runtime telemetry. Recommendation logic is deterministic and auditable, but not autonomous remediation. These limitations do not undercut the systems contribution; they define it. The paper evaluates whether the implemented pipeline is coherent, reproducible, and internally validated as a post-billing FinOps decision engine, not whether it has already exhausted the broader space of production cloud cost optimization.

VIII. CONCLUSION AND FUTURE WORK

This paper addressed the core systems problem behind multi-cloud FinOps decision support: provider billing exports are not directly comparable, commitment semantics distort raw workload cost, and actionable optimization requires accountable attribution rather than descriptive spend visibility alone. The implemented system responds with a canonical Unified Cost Allocation Schema, commitment-aware Net Effective Cost modeling, explicit shared-cost allocation semantics, and a deterministic decision pipeline that materializes waste signals, ranked recommendations, month-end forecasts, and lifecycle-ready validation outputs.

The current repository state provides meaningful internal evidence for this design. The synthetic benchmark materializes 18,992 normalized billing rows across AWS, Azure, and GCP, preserves Azure shared-cost fan-out exactly in each observed month, produces 95 persisted recommendations, and achieves forecast backtest quality of \$7.86 mean absolute error, 2.98% mean absolute percentage error, and a 95.59% within-10-percent rate. These results support the claim that the system is reproducible, auditable, and sufficiently coherent for workshop or conference-style systems evaluation.

The paper should nevertheless state its boundaries clearly. The benchmark is synthetic rather than enterprise production data. Idle compute detection is based on billing-side proxies rather than runtime telemetry. The lifecycle layer is implemented, but the current benchmark state is still pre-realization: all recommendations remain in the `recommended` state, so realized-savings validation is not yet mature. The system also stops at decision support and validation instrumentation; it does not perform autonomous remediation.

These limitations lead directly to future work. The most credible next step is enterprise validation on real multi-cloud billing exports with controlled access to ground-truth financial review. A second direction is telemetry-backed waste detection that combines billing semantics with utilization signals from compute, storage, and orchestration layers. A third is stronger closed-loop calibration, including richer implemented and verified lifecycle outcomes for recommendation classes such as `resize_down` and `remove_resource`. A fourth is workflow integration with ticketing, approval, and governance systems so that recommendations can move beyond local persistence into operational execution. Finally, the deterministic methods here provide a useful baseline for studying when more advanced forecasting or reasoning models improve accuracy without sacrificing auditability.

REFERENCES

- [1] Amazon Web Services, “Aws cost and usage report data dictionary,” <https://docs.aws.amazon.com/cur/latest/userguide/Lineitem-columns.html>, 2026, accessed 2026-05-29.
- [2] Microsoft, “View amortized costs in azure cost management,” <https://learn.microsoft.com/azure/cost-management-billing/>, 2026, accessed 2026-05-29.
- [3] Google Cloud, “Cloud billing export to bigquery,” <https://cloud.google.com/billing/docs/how-to/export-data-bigquery>, 2026, accessed 2026-05-29.
- [4] FinOps Foundation, “Finops framework,” <https://www.finops.org/framework/>, 2026, accessed 2026-05-29.
- [5] —, “Inform, optimize, and operate,” <https://www.finops.org/framework/capabilities/>, 2026, accessed 2026-05-29.
- [6] H. Baars and H.-G. Kemper, “Chargeback for cloud services,” in *International Conference on Wirtschaftsinformatik*, 2014.
- [7] R. Gupta and N. Gupta, “Streamlining multi-cloud management,” *International Journal of Cloud Applications*, vol. 4, no. 2, pp. 15–27, 2016.
- [8] M. Imran *et al.*, “Multi-cloud resource and cost management: A survey,” *Journal of Network and Computer Applications*, vol. 168, p. 102760, 2020.
- [9] Amazon Web Services, “Reservation and savings plans columns in aws cur,” <https://docs.aws.amazon.com/cur/latest/userguide/reservation-columns.html>, 2026, accessed 2026-05-29.
- [10] Microsoft, “Understand azure reservation usage,” <https://learn.microsoft.com/azure/cost-management-billing/reservations/>, 2026, accessed 2026-05-29.
- [11] Google Cloud, “Committed use discounts overview,” <https://cloud.google.com/compute/docs/instances/committed-use-discounts-overview>, 2026, accessed 2026-05-29.
- [12] FinOps Foundation, “Finops open cost and usage specification (focus),” <https://focus.finops.org/>, 2026, accessed 2026-05-29.
- [13] X. Liu *et al.*, “Cloud cost optimization techniques: A survey,” *ACM Computing Surveys*, vol. 55, no. 11, pp. 1–36, 2023.